

Rust & Python : le menu parfait ?

2026-09-29

1.

Mise en bouche

Pourquoi mettre du 🦀 dans du 🐍 ?

1.1 Ce qu'il y a déjà dans votre assiette

1.1 Ce qu'il y a déjà dans votre assiette

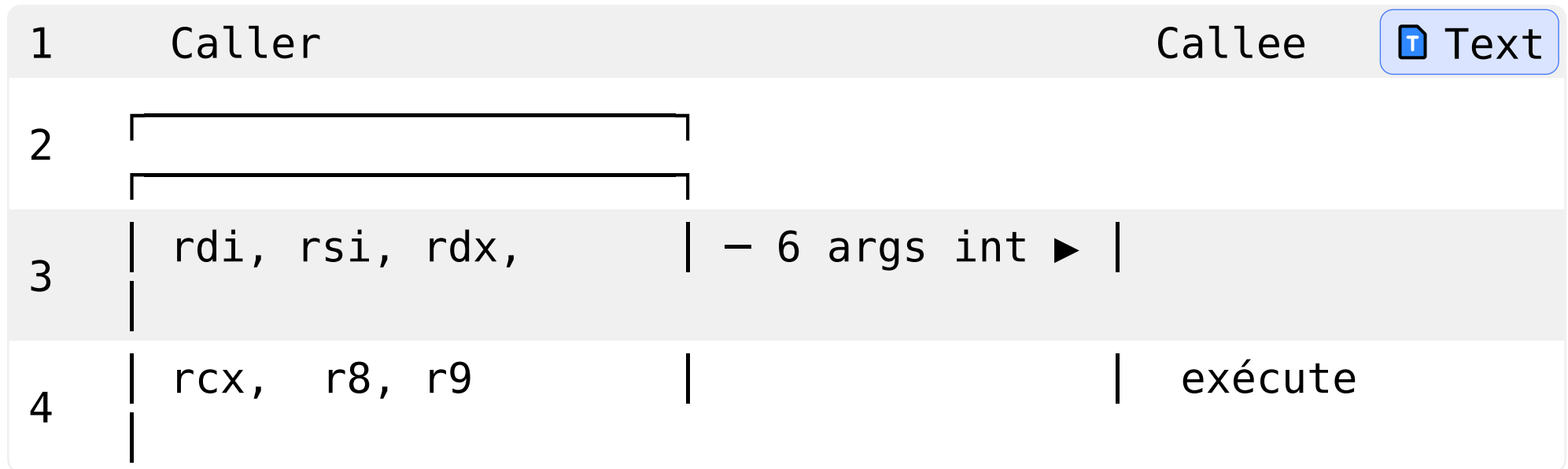
- 🐼 Polars — DataFrame
- 🧹 Ruff — linter
- 📦 uv — packaging
- ✅ pydantic-core — validation
- 📄 tokenizers — NLP
- ⚡ orjson — JSON

→ La stack Python qui va vite est **écrite en Rust**.

1.2 La cuisine centrale

1.2 La cuisine centrale

Tout le monde parle **C** en sortie de compilation.



5	xmm0...xmm7	- flottants →	
6	pile (au-delà)		rax = résultat
7		— call →	ret
8	rax ← retour	← return —	
9			

Ce qui ne passe pas le comptoir : noms manglés (extern "C"), layout des structs (repr(C)), chaînes (\0 vs (ptr, len)).

1.3 Les ustensiles

1.3 Les ustensiles

Approche	Pour	Contre
ctypes	zéro compilation	lent, aucune sûreté
cffi	ABI ou API mode	une dépendance de plus
C-API	perf maximale	tout à la main, en C
PyO3	ergonomie Rust + perf C-API	il faut compiler du Rust

→ L'apport de PyO3, ce n'est pas l'appel. C'est **tout le reste** : conversions de types, exceptions, docstrings, signatures — **dans les deux sens**.

2.

La mise en place

PyO3 + Maturin, et on ouvre

2.1 Les ingrédients

2.1 Les ingrédients

```
1 # Cargo.toml toml  
2 [lib]  
3 crate-type = ["cdylib"]  
4  
5 [dependencies]  
6 pyo3 = { version = "0.29", features = ["abi3-py38"] }
```

```
1 # pyproject.toml toml
```

```
2 [build-system]
3 requires = ["maturin>=1.9,<2.0"]
4 build-backend = "maturin"
```

```
1 $ maturin develop          # compile + installe
   dans le venv
2 $ maturin build --release  # produit une wheel
```



2.3 Le plat servi

2.3 Le plat servi

```
1  use pyo3::prelude::*;
2
3  /// Convertit du Markdown en HTML.
4  #[pyfunction]
5  #[pyo3(signature = (input, *, github_flavored = false))]
6  fn markdown_to_html(input: &str, github_flavored: bool)
   -> String {
7      let mut options = comrak::Options::default();
```



```
8      options.extension.strikethrough = github_flavored;
9      options.extension.table = github_flavored;
10     comrak::markdown_to_html(input, &options)
11 }
12
13 /// Module Python `rusty_md`.
14 #[pymodule]
15 fn rusty_md(m: &Bound<'_, PyModule>) -> PyResult<()> {
16     m.add_function(wrap_pyfunction!(markdown_to_html,
17     m)?)?;
17     Ok(())
```

```
18 }
```

```
19
```

```
1 >>> markdown_to_html(42)
```

 Python

```
2 TypeError: 'int' object is not an instance of 'str'
```

```
3 while processing 'input'
```

2.4 Le service inversé

2.4 Le service inversé

Un binaire **Rust** qui embarque un interpréteur **Python**.

```
1 fn main() -> PyResult<()> {  
2     Python::attach(|py| {  
3         let math = py.import("math"?;  
4         let pi: f64 = mathgetattr("pi"?).extract()?;  
5         println!("π = {pi}");  
6     }
```



```
7         py.run(c"print('Bonjour depuis Python')", None,  
                None)  
8     })  
9 }
```

```
1 pyo3 = { version = "0.29", features = ["auto-  
    initialize"] }
```

toml

3.

Le coup de feu

Ce que ça donne quand la salle est pleine

3.1 Le plat signature

3.1 Le plat signature

Cette présentation est pilotée par toboggan-py.

```
1  toboggan-core/      ← logique métier pure (no_std  
   capable)  
2  toboggan-client/   ← client HTTP + WebSocket, async  
   (tokio)  
3  toboggan-py/       ← bindings Py03 (cdylib, abi3-py38)  
4      └─ src/  
5      └─ lib.rs       #[pymodule]
```

 Text

```
6      └─ toboggan.rs    #[pyclass] Toboggan
7      └─ talk.rs, slides.rs, state.rs, ...
8  toboggan_py.pyi    ← type stubs (IDE, mypy)
```

→ **Le même cœur métier**, exposé en CLI, Web, Mobile, Python.

3.2 Le passe-plat asynchrone

3.2 Le passe-plat asynchrone

Rust est **async**. Python, ici, est **synchrone**.

```
1  #[pyclass]
2  pub struct Toboggan {
3      rt: Runtime,                // le runtime tokio
    vit dans la classe
4      tx: UnboundedSender<Command>, // canal vers la
    tâche de fond
```



```
5      state: Arc<RwLock<TState>>,      // état tenu à jour
      en tâche de fond
6  }
7
8  #[pymethods]
9  impl Toboggan {
10     #[getter]
11     pub fn state(&self) -> PyResult<State> {
12         let state = Arc::clone(&self.state);
13         Ok(State(self.rt.block_on(async {
```

```
14         TState::clone(&*state.read().await)    // on
           clone, on ne prête pas le guard
15     })))
16 }
17
18     pub fn next(&self)
19     { self.send(Command::NextSlide); }    // fire-and-
      forget
19 }
```

3.3 Deux façons de servir

3.3 Deux façons de servir

```
1 // 1 – Runtime interne, dans #[pymethods].  
  Python : client.fetch(url)  
2 fn fetch(&self, url: &str) -> PyResult<String> {  
3     self.rt.block_on(async  
        { request::get(url).await?.text().await })  
4         .map_err(|e|  
            PyRuntimeError::new_err(e.to_string()))  
5 }
```



```
1 // 2 – pyo3-async-runtimes. Python : await  
  fetch(url)
```



```
2 #[pyfunction]
```

```
3 fn fetch(py: Python<'_>, url: String) ->  
  PyResult<Bound<'_, PyAny>> {
```

```
4     pyo3_async_runtimes::tokio::future_into_py(py, async  
        move {
```

```
5         let rsp =  
            request::get(&url).await.map_err(to_py)?;
```

```
6         rsp.text().await.map_err(to_py)
```

```
7     })
```

```
8 }
```

→ Choisir selon **qui** consomme : un notebook, ou un service asyncio.

3.4 Suivre la commande

3.4 Suivre la commande

```
1 Python (otel-python)
   (opentelemetry-rust)
```

Rust

 Text

```
2 _____
   _____
```

```
3 trace_id: ABC
```

```
trace_id: ???
```

```
4 span_id: 001
```

```
span_id: ???
```

```
5 |
```

```
|
```



Solution : sérialiser le **W3C TraceContext** à la frontière.

1 Python : `propagate.inject(carrier)` →
`{"traceparent": "00-ABC-001-01"}`

 Text

2 Rust : `Extractor` → reconstruit le Context côté
`opentelemetry-rust`

4.

L'addition

Ce que ça coûte, ce que ça rapporte

4.1 Allergènes

4.1 Allergènes

-  **Les .pyi sont écrits à la main** — et ils mentent vite : sur toboggan-py, 7 méthodes manquantes et un variant d'enum disparu
-  **Pas de génériques** — `#[pyclass] struct Container<T>` ne compile pas : il faut monomorphiser
-  **Pas de traits → protocoles** — `Iterator` ne devient pas itérable : il faut écrire `__iter__` / `__next__`
-  **Pas d'héritage en diamant** Python → Rust → Python

-  **Le GIL reste pris** pendant un `block_on` : un aller-retour réseau gèle les autres threads Python
- Presque tout se contourne. Mais ça se **paie en wrapper**.

4.2 Pourboires

4.2 Pourboires

-  **Réduire la surface FFI** — chaque franchissement se paie
-  **Batcher** — un appel pour 1 M d'éléments, pas 1 M d'appels
-  **Un seul runtime tokio** — static RT: `OnceLock<Runtime>`
-  **`py.detach(...)`** sur les sections CPU-bound et les I/O bloquantes
-  **Tester des deux côtés** — `pytest` *et* `cargo test`
-  **`py-spy`** — le flame graph voit les frames Rust
-  **Générer les stubs** plutôt que les écrire

4.3 Les combos du menu

4.3 Les combos du menu

1. **Le moteur** — garder l'API Python, remplacer le cœur par du Rust
Polars, pydantic-core, Ruff
2. **L'API polyglotte** — un cœur Rust, exposé partout *OpenDAL · xberg* :
1 core, 15 bindings
3. **Proto Python → Rust** — itérer vite, optimiser quand le besoin apparaît
4. 🤔 **A-t-on encore besoin de Python ?** *polars, candle, evcxr...*
mais l'écosystème data reste là-bas

Des questions ?

4.4 La carte

Des questions ?

- **Outils** — [pyo3.rs](#) · [maturin.rs](#) · écosystème PyO3 · [pyo3-async-runtimes](#) · [rust-numpy](#) · [py-spy](#)
- **Les talks de David Hewitt**, mainteneur PyO3 — [PyO3 en profondeur](#) · [Free-threaded Python](#) · [5 ans de Rust dans Python](#) · [ITW en français](#)
- **Le code de cette conf** — github.com/ilaborie/toboggan (toboggan-py)